Rapporto tecnico per confronto con lo Studio IA

Rilevazione dell’oscillazione corporea nell’app Analogista Virtuale

Progetto: Analogista Virtuale di Max Pisani
Repository: `Analogista/1.0-Analogista-Virtuale`
Applicazione pubblicata: analogistavirtuale2027.ai.studio
Area analizzata: fotocamera, rilevazione posa, calibrazione, classificazione avanti/indietro, svolgimento dei test e compatibilità con dispositivi mobili.
Obiettivo: fornire una base tecnica di confronto con lo Studio IA che ha sviluppato l’app, senza modificare il significato metodologico dei risultati.

---

1. Requisiti confermati dal committente

Le seguenti decisioni devono essere considerate vincolanti durante qualsiasi eventuale revisione:

1. Il significato attuale deve rimanere invariato:
   - movimento in avanti = `SI`;
   - movimento all’indietro = `NO`.
2. La priorità di compatibilità è Android, soprattutto smartphone utilizzati tramite browser mobile.
3. La fotocamera deve inquadrare il busto dalla vita alla testa.
4. Quando il segnale non è sufficientemente affidabile, il risultato deve essere `NON_RILEVATO`, non un SI o un NO forzato.
5. Le migliorie devono essere snelle e non devono alterare il metodo interpretativo dei test.
6. L’elaborazione della rilevazione dovrebbe rimanere locale nel browser, senza salvataggio automatico del video grezzo.

Questo documento non propone di sostituire la logica analogica dell’app. Propone esclusivamente di rendere più affidabile la rilevazione tecnica del segnale corporeo e più chiaro il percorso operativo.

---

2. Architettura attuale rilevata

La rilevazione è centralizzata principalmente nei seguenti file:

- `src/components/CameraView.tsx`
- `src/services/MotionDetectionService.ts`
- `src/services/AdvancedVoiceService.ts`

`CameraView` gestisce:

- richiesta della fotocamera tramite `navigator.mediaDevices.getUserMedia`;
- visualizzazione del video;
- canvas sovrapposto con alcuni landmark;
- conto alla rovescia della calibrazione;
- indicatori visivi `ATTESA`, `PRONTO` e `CALIBRAZIONE`;
- ricezione degli eventi di movimento;
- visualizzazione del risultato `SI` o `NO`.

`MotionDetectionService` gestisce:

- caricamento di MediaPipe Pose Landmarker;
- caricamento del modello `pose_landmarker_lite`;
- analisi video in modalità `VIDEO`;
- estrazione di naso e spalle;
- calcolo della misura di profondità apparente;
- smoothing EMA;
- soglia, isteresi e debounce;
- emissione degli eventi `movementDetected`, `depthUpdate` e `landmarksProcessed`.

`AdvancedVoiceService` gestisce:

- voce guida;
- avvio e arresto del rilevamento durante una domanda;
- attesa di una risposta;
- conversione `forward` in `SI`;
- conversione `backward` in `NO`;
- timeout in `NON_RILEVATO`.

La `CameraView` viene riutilizzata in più test, tra cui:

- calibrazione;
- Test Induttore;
- Test Nome;
- Test Punti Distonici;
- Test Sigilli;
- Test Linea Temporale;
- Test Testimone Chiave;
- Test T.A.P.

Questa centralizzazione è positiva perché consente di correggere il nucleo della rilevazione senza riscrivere tutte le schermate.

---

3. Funzionamento tecnico attuale

Il servizio considera tre landmark principali:

- naso: indice MediaPipe `0`;
- spalla sinistra: indice `11`;
- spalla destra: indice `12`.

La misura denominata profondità viene calcolata in questo modo concettuale:

```
Plain Text

misura = larghezza delle spalle × 0,7
       + distanza tra naso e centro delle spalle × 0,3
```

Il valore viene filtrato con una media mobile esponenziale. Dopo la calibrazione viene calcolato lo scostamento rispetto al valore neutro:

```
Plain Text

scostamento = valore corrente filtrato - valore neutro
```

La classificazione attuale è:

```
Plain Text

scostamento oltre soglia positiva  -> forward -> SI
scostamento oltre soglia negativa  -> backward -> NO
```

```
scostamento vicino al neutro       -> none
```

È inoltre presente un meccanismo di isteresi per evitare continui cambi tra stati e un debounce audio di circa 600 millisecondi.

Il servizio vocale aspetta una risposta per circa 12 secondi. Se non arriva un evento valido, restituisce `NON_RILEVATO` e comunica che non è stato rilevato un movimento.

---

4. Criticità prioritaria: sincronizzazione della calibrazione

Situazione individuata

Nel componente `CameraView`, al termine del conto alla rovescia venivano eseguite queste operazioni:

Plain Text

```
motionService.current?.startDetection();
motionService.current?.calibrate();
```

Nel servizio `MotionDetectionService`, `startDetection()` esegue anche:

Plain Text

```
this.currentSmoothedDepth = null;
```

Il primo nuovo fotogramma viene però elaborato solo nel successivo ciclo di `requestAnimationFrame`. Per questo motivo `calibrate()` può essere chiamata quando il nuovo valore filtrato non è ancora disponibile.

Possibili conseguenze

- la calibrazione può non aggiornare il valore neutro;
- il valore neutro può restare a zero;
- il valore precedente può essere utilizzato in una ricalibrazione;
- il primo movimento successivo può essere classificato in modo errato;
- il problema può manifestarsi in modo intermittente e sembrare dipendente dal dispositivo.

Questa criticità è particolarmente importante perché influenza tutti i test che dipendono dalla rilevazione avanti/indietro.

Correzione proposta

La calibrazione dovrebbe raccogliere un campione per una breve finestra temporale, per esempio 1–2 secondi, invece di utilizzare un solo fotogramma.

Procedura consigliata:

1. fermare la normale rilevazione;
2. iniziare una finestra di calibrazione;
3. raccogliere solo fotogrammi con landmark validi;
4. calcolare una mediana o una media robusta;
5. verificare che esista un numero minimo di campioni;
6. salvare il risultato come neutro;
7. riattivare la rilevazione normale;
8. se i campioni non sono sufficienti, restituire un errore di calibrazione e chiedere di riprovare.

Nel clone locale del repository è stata preparata una correzione con calibrazione multi-campione. La modifica non è stata ancora pubblicata su GitHub e non cambia la corrispondenza `SI` / `NO`.

---

5. Criticità di inizializzazione di MediaPipe

Attualmente `MotionDetectionService.initialize()` intercetta l'errore di MediaPipe ma non lo propaga chiaramente alla `CameraView`. La schermata può quindi rischiare di apparire pronta anche quando il modello non è realmente disponibile.

Il comportamento consigliato è:

```
Plain Text

MediaPipe inizializzato correttamente -> PRONTO
MediaPipe non inizializzato            -> ERRORE / RIPROVA
```

Non dovrebbe essere possibile avviare un test se:

- il modello non è stato caricato;
- il delegate non è disponibile;
- il browser non supporta la tecnologia necessaria;
- il video non ha ancora dimensioni valide;

- non sono stati rilevati landmark affidabili.

Nel codice preparato localmente, `initialize()` restituisce ora un valore booleano e `CameraView` non imposta `PRONTO` quando l'inizializzazione fallisce.

---

6. Compatibilità Android

La priorità indicata è Android. I principali aspetti da verificare sono i seguenti.

6.1 Browser e permessi

La fotocamera deve essere utilizzata esclusivamente in contesto HTTPS. Il codice dovrebbe gestire separatamente:

- permesso negato;
- permesso revocato dalle impostazioni del browser;
- fotocamera già utilizzata da un'altra app;
- dispositivo senza fotocamera disponibile;
- browser non compatibile;
- pagina caricata in un contesto non sicuro.

Il messaggio generico "Errore hardware fotocamera" non è sufficiente per aiutare l'utente a risolvere il problema.

6.2 Vincoli video

Per Android conviene mantenere vincoli moderati, per esempio:

Plain Text

```
{
  video: {
    facingMode: 'user',
    width: { ideal: 1280 },
    height: { ideal: 720 },
    frameRate: { ideal: 24, max: 30 }
  },
  audio: false
}
```

Se questi vincoli falliscono, il fallback a `video: true` è corretto, ma sarebbe utile registrare tecnicamente quale configurazione è stata ottenuta.

6.3 GPU e CPU

Il codice chiede il delegate GPU. Su alcuni dispositivi Android, WebGL o il delegate GPU possono non essere disponibili o possono comportarsi in modo instabile.

È consigliabile:

1. tentare il delegate GPU;
2. se fallisce, provare automaticamente il delegate CPU;
3. comunicare all' utente solo un eventuale rallentamento, non un errore tecnico complesso.

Nel clone locale è stato predisposto questo fallback GPU/CPU.

6.4 Prestazioni

`detectForVideo()` viene chiamato dentro `requestAnimationFrame`. Sui telefoni meno potenti ciò può occupare troppo il thread principale.

Suggerimenti leggeri:

- limitare l' analisi a 20–25 FPS su Android;
- mantenere il video fluido anche quando l' analisi viene ridotta;
- ignorare fotogrammi troppo ravvicinati;
- misurare il tempo impiegato da MediaPipe;
- mostrare `Qualità bassa` o `Dispositivo lento` quando necessario.

Non è necessario introdurre subito un' architettura complessa con Web Worker. È sufficiente un limite temporale tra due chiamate successive.

---

7. Qualità dei landmark e falso movimento

La formula attuale non usa direttamente la coordinata `z` fornita da MediaPipe. Utilizza invece una variazione geometrica apparente nel piano 2D.

Questo può essere sensibile a:

- rotazione del tronco;
- inclinazione della testa;
- abbassamento o sollevamento del mento;
- movimento laterale;
- variazioni di luce;

- persona troppo vicina o troppo lontana;
- perdita temporanea di uno dei landmark;
- rilevamento errato della spalla;
- sfondo confuso o presenza di altre persone.

Non si propone di cambiare subito la formula, perché ciò potrebbe modificare il comportamento metodologico. Prima è preferibile aggiungere controlli di qualità.

Controlli suggeriti

Prima di usare un fotogramma nel calcolo:

- verificare la presenza dei tre landmark necessari;
- verificare `visibility` o `presence` quando disponibili;
- scartare il fotogramma se un punto è troppo poco affidabile;
- verificare che la distanza tra le spalle sia plausibile;
- verificare che il volto e il busto non siano fuori dall' inquadratura.

Se i fotogrammi validi non sono sufficienti, il sistema deve produrre `NON_RILEVATO`.

---

8. Raccomandazione sul risultato "NON_RILEVATO"

La preferenza confermata è non trasformare un segnale debole in un risultato SI o NO.

La classificazione consigliata è:

```
Plain Text

SI              movimento avanti valido e confermato
NO              movimento indietro valido e confermato
NON_RILEVATO    segnale assente, debole, instabile o tecnicamente non valido
```

Un risultato dovrebbe essere accettato solo se:

- supera la soglia;
- rimane coerente per più fotogrammi;
- i landmark sono affidabili;
- non è immediatamente seguito da perdita di tracking;
- non deriva da una calibrazione fallita.

Quando il risultato è `NON_RILEVATO`, l' app dovrebbe offrire:

- un messaggio chiaro;
- un pulsante `Riprova test`;
- la possibilità di ripetere solo la domanda corrente;
- conservazione dei risultati già ottenuti negli step precedenti.

Non dovrebbe avanzare automaticamente con un dato incerto.

---

9. Miglioramento dello svolgimento operativo dei test

9.1 Preparazione iniziale

Prima di chiedere la calibrazione, l’ app dovrebbe mostrare una breve lista:

1. appoggia lo smartphone in posizione stabile;
2. usa preferibilmente orientamento verticale;
3. inquadra il busto dalla vita alla testa;
4. mantieni luce frontale sufficiente;
5. evita una finestra molto luminosa alle spalle;
6. rimani fermo durante la calibrazione.

9.2 Controllo automatico della posizione

Prima del conto alla rovescia si potrebbe verificare:

- busto centrato;
- spalle e volto visibili;
- distanza adeguata;
- luminosità sufficiente;
- tracking stabile per alcuni fotogrammi.

Messaggi semplici sarebbero più efficaci di un errore generico:

- “Centra il busto”;
- “Allontanati leggermente”;
- “Avvicinati leggermente”;
- “Aumenta la luce”;
- “Rimani fermo”.

9.3 Calibrazione

Sequenza consigliata:

```
Plain Text

Preparati
3... 2... 1...
Rimani fermo
Acquisizione posizione neutra
Calibrazione completata
```

Durante la calibrazione non devono essere emessi eventi SI o NO.

9.4 Domanda e risposta

Dopo la voce guida si consiglia una breve pausa tecnica di circa 500–800 millisecondi. Poi può iniziare la finestra di rilevazione.

Una finestra di 8–12 secondi è coerente con l' attuale timeout di 12 secondi, ma il risultato dovrebbe essere confermato solo quando il movimento è stabile.

9.5 Ritorno al neutro

Tra una domanda e l' altra, soprattutto nel Test Induttore, è utile inserire 1–2 secondi per tornare alla posizione neutra. Questo riduce il rischio che il movimento precedente venga interpretato come risposta alla domanda successiva.

9.6 Test Induttore

Poiché il test analizza prima la mano destra e poi la sinistra, si suggerisce di rendere espliciti questi passaggi:

```
Plain Text

Calibrazione
Domanda mano destra
Risultato o NON_RILEVATO
Ritorno al neutro
Pausa
Domanda mano sinistra
Risultato o NON_RILEVATO
Calcolo della combinazione finale
```

Se una delle due risposte è `NON_RILEVATO`, il risultato dell' Induttore dovrebbe rimanere non definito e il test dovrebbe proporre la ripetizione.

9.7 Test T.A.P.

Per il Test T.A.P., ogni simbolo dovrebbe avere:

- propria indicazione di stato;
- propria calibrazione o riferimento chiaramente dichiarato;
- risultato SI/NO/NON_RILEVATO;
- indicazione del ciclo in corso;
- nessuna contaminazione dal movimento rilevato sul simbolo precedente.

---

10. Miglioramenti all’interfaccia dell’app

Sono consigliabili modifiche semplici, senza ridisegnare l’app:

- mostrare `Test 3 di 8` o un indicatore di avanzamento;
- distinguere `completato` da `completato con risultato non rilevato`;
- permettere di ripetere solo l’ultimo test;
- conservare il percorso in `localStorage` o IndexedDB;
- recuperare la sessione dopo un refresh accidentale;
- non perdere nome, età e risposte precedenti quando la fotocamera viene riavviata;
- usare un pulsante `Riprova fotocamera` invece di `Ricarica App`;
- mostrare una conferma quando la fotocamera è stata effettivamente rilasciata.

Il salvataggio locale dei dati di percorso dovrebbe essere accompagnato da una gestione chiara della privacy e da un comando per cancellare la sessione.

---

11. Privacy e sicurezza

L’app comunica già all’utente che non costituisce strumento di diagnosi o cura. Questa avvertenza deve rimanere visibile anche nei risultati.

Per la rilevazione tramite fotocamera si consiglia di:

- elaborare il flusso localmente nel browser;
- non salvare automaticamente il video;
- non inviare fotogrammi al server salvo esplicito consenso;
- salvare al massimo risultato, qualità tecnica e dati di sessione necessari;
- spiegare chiaramente quando una sessione viene salvata su Firebase;

- separare dati tecnici anonimi da dati personali;
- consentire la cancellazione dei dati locali.

I dati come nome, età e risultati dei test devono essere trattati come dati personali e non dovrebbero essere inclusi nei log tecnici del browser o inviati a servizi esterni senza una motivazione esplicita.

---

12. Compatibilità PWA e rete

Il progetto usa una configurazione PWA. Tuttavia il modello MediaPipe e i file WASM vengono richiesti da URL esterni.

Da verificare:

- comportamento al primo caricamento senza rete;
- comportamento quando la rete cade dopo l’ apertura dell’ app;
- disponibilità del modello dopo il primo caricamento;
- eventuale caching controllato del modello;
- compatibilità delle policy CORS;
- tempi di caricamento su rete mobile lenta.

Una strategia prudente è:

1. mantenere il caricamento remoto nella prima fase;
2. mostrare uno stato “Caricamento motore di rilevazione”;
3. non dichiarare la fotocamera pronta prima del caricamento completo;
4. valutare successivamente il caching locale del modello.

---

13. Piano di intervento leggero consigliato

Fase A — Correzione del nucleo

- calibrazione multi-campione;
- verifica reale dell’ inizializzazione MediaPipe;
- fallback GPU/CPU;
- risultato `NON_RILEVATO` per tracking insufficiente.

Fase B — Robustezza Android

- messaggi distinti per i principali errori fotocamera;
- frame rate massimo per il rilevamento;
- controllo dei landmark e della luminosità;
- pulsante di riprova senza ricaricare la pagina.

Fase C — Esperienza di test

- istruzioni iniziali più chiare;
- stato di posizione corretta;
- pausa tra domanda e rilevazione;
- ritorno al neutro tra due domande;
- ripetizione del solo test non rilevato.

Fase D — Validazione sul campo

Eseguire prove almeno su:

- Android economico con Chrome;
- Android recente con Chrome;
- Android Samsung con Samsung Internet;
- tablet Android;
- desktop Chrome come riferimento.

Per ogni dispositivo registrare:

- modello;
- versione Android;
- browser e versione;
- orientamento;
- illuminazione;
- distanza dalla fotocamera;
- tempo di caricamento MediaPipe;
- numero di risposte corrette;
- numero di `NON_RILEVATO`;
- eventuali blocchi o rallentamenti.

14. Criteri di accettazione suggeriti

Una revisione dovrebbe essere considerata tecnicamente riuscita se:

1. la calibrazione non salva il neutro prima di avere campioni sufficienti;
2. un errore MediaPipe non produce lo stato `PRONTO`;
3. un dispositivo Android senza GPU compatibile può tentare il fallback CPU;
4. il significato `avanti = SI` e `indietro = NO` resta invariato;
5. i segnali deboli producono `NON_RILEVATO`;
6. un test non valido può essere ripetuto senza perdere il percorso precedente;
7. la fotocamera viene rilasciata correttamente quando si cambia schermata;
8. la build di produzione termina correttamente;
9. i dati video non vengono salvati o inviati automaticamente;
10. i risultati restano comprensibili per l'utente senza esporre dettagli tecnici inutili.

---

15. Domande da sottoporre allo Studio IA

1. La calibrazione attuale è stata progettata per usare un singolo fotogramma o una finestra di campionamento?
2. È previsto un controllo di stabilità durante la calibrazione?
3. Quali modelli Android e browser sono stati utilizzati per la validazione?
4. È stato verificato il comportamento quando il delegate GPU di MediaPipe non è disponibile?
5. Qual è il criterio esatto per dichiarare affidabile un landmark?
6. Come viene distinto un movimento reale dalla perdita temporanea del tracking?
7. Qual è la ragione tecnica della formula basata su larghezza delle spalle e distanza naso-spalle invece dell'uso della coordinata `z`?
8. Sono state effettuate prove con luce bassa, controluce, occhiali e abiti scuri?
9. Il risultato `NON_RILEVATO` deve essere mostrato come esito neutro, errore tecnico o richiesta di ripetizione?
10. Tra due domande viene azzerato o mantenuto il riferimento neutro?
11. Nel Test Induttore, il movimento della mano destra può influenzare la successiva misurazione della mano sinistra?

12. Il modello MediaPipe e i file WASM vengono memorizzati localmente dopo il primo caricamento?
13. Quali dati vengono inviati a Firebase e quali restano esclusivamente nel dispositivo?
14. Esiste una procedura di ripristino se l'utente aggiorna la pagina durante un test?
15. È possibile introdurre i miglioramenti di robustezza senza modificare le regole interpretative del metodo?

---

16. Stato delle modifiche preparate localmente

Nel clone locale del repository sono state preparate modifiche limitate a:

- `src/components/CameraView.tsx`;
- `src/services/MotionDetectionService.ts`.

Le modifiche preparate:

- sostituiscono la calibrazione istantanea con una calibrazione multi-campione;
- restituiscono un fallimento se non vengono acquisiti abbastanza punti;
- impediscono lo stato `PRONTO` quando MediaPipe non è inizializzato;
- allineano la versione WASM alla dipendenza del progetto;
- tentano GPU e poi CPU come fallback.

La build Vite e il controllo TypeScript sono riusciti. Il lint completo non è stato eseguito perché il repository presenta già un'incoerenza tra `package.json`, `package-lock.json` e la versione di ESLint. Questo problema è indipendente dalla logica della fotocamera e andrebbe risolto separatamente, evitando di mescolarlo alla correzione funzionale.

Le modifiche non sono state ancora pubblicate su GitHub. Prima della pubblicazione è preferibile confrontare questo rapporto con lo Studio IA e verificare che la calibrazione multi-campione sia compatibile con il comportamento metodologico desiderato.

Conclusione

L'architettura attuale è riutilizzabile e la centralizzazione in `CameraView` e `MotionDetectionService` consente miglioramenti mirati senza riscrivere l'app. La criticità principale da verificare è la sincronizzazione della calibrazione, perché può influenzare tutti i test successivi.

La linea consigliata è conservativa: mantenere invariati i significati `SI` e `NO`, migliorare la qualità tecnica del segnale, usare `NON_RILEVATO` quando i dati non sono sufficienti e

privilegiare la stabilità su Android prima di introdurre modifiche più complesse.